



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

Department of Computer Science and Engineering

CS23334 Fundamentals of Data Science Lab

III semester II Year (2023R)

Name of the Student: DEEPTHI DHARUNYA R

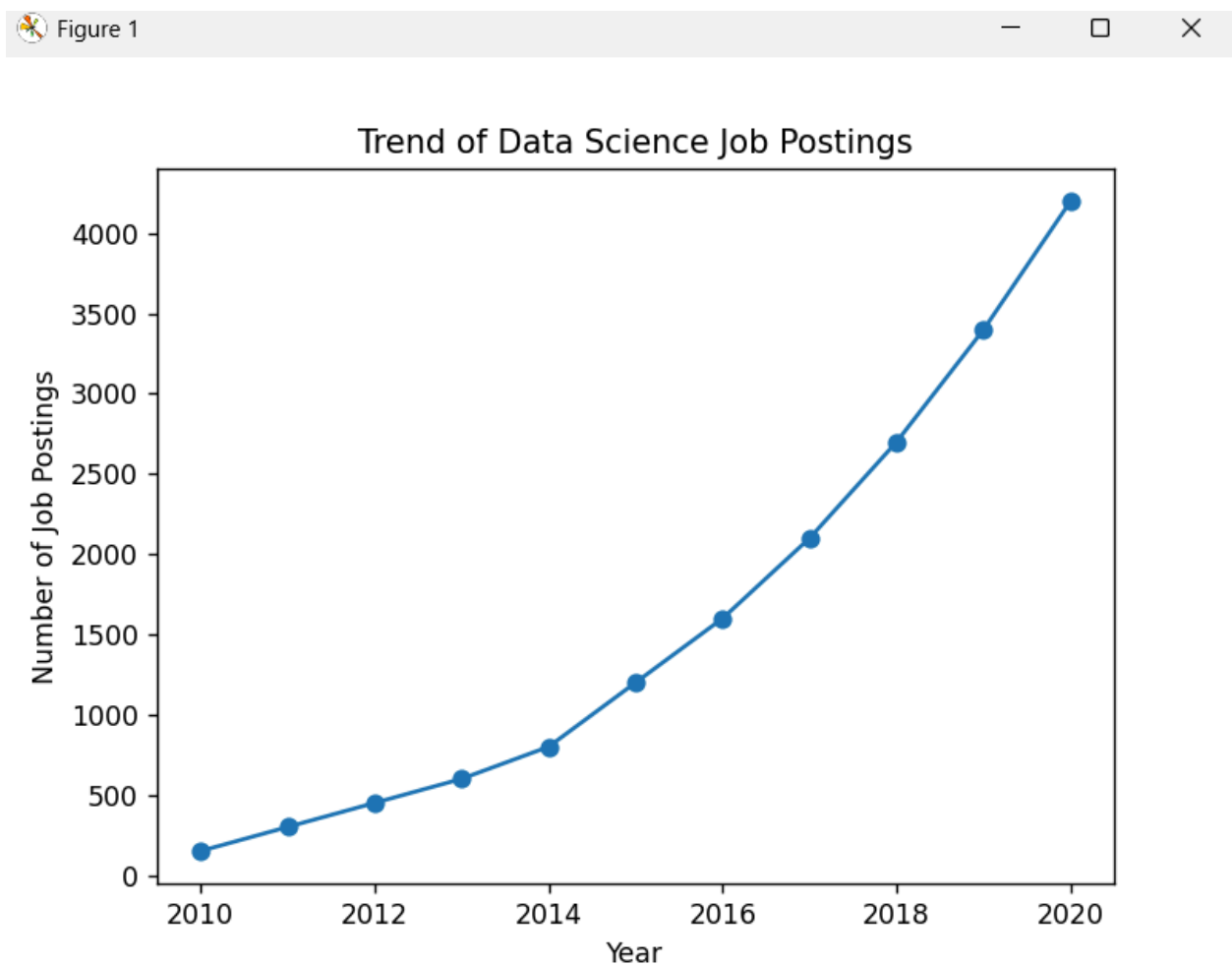
Register Number: 2116240701105

EXPERIMENT 01

Analyze the trends of datascience job posting

```
import pandas as pd
import matplotlib.pyplot as plt
data = {'Year': list(range(2010, 2021)),
        'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400, 4200]}

df = pd.DataFrame(data)
plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```



EXPERIMENT 02

```
*fds.py - C:/Users/DEEPTHI/OneDrive/Desktop/New folder/fds.py (3.14.0)*
File Edit Format Run Options Window Help

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Load the data into a pandas DataFrame
file_path=r"C:\Users\DEEPTHI\OneDrive\Desktop\fds obs"
df = pd.read_csv(file_path)
# Display the first few rows of the DataFrame
print(df.head())
# Check for missing values
print(df.isnull().sum())
# Fill or drop missing values if necessary
df['Sales'].fillna(df['Sales'].mean(), inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
# Summary statistics
print(df.describe())
# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()
print(product_summary)

# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('SalesOver Time')
plt.show()

# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc=np.sum, fill_value=0)
print(pivot_table)
# Correlation matrix
correlation_matrix = df.corr()
print(correlation_matrix)
# Heatmap of the correlation matrix
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

	Date	Product	Sales	Quantity	Region
0	2023-01-01	Product A	200	4	North
1	2023-01-02	Product B	150	3	South
2	2023-01-03	Product A	220	5	North
3	2023-01-04	Product C	300	6	East
4	2023-01-05	Product B	180	4	West

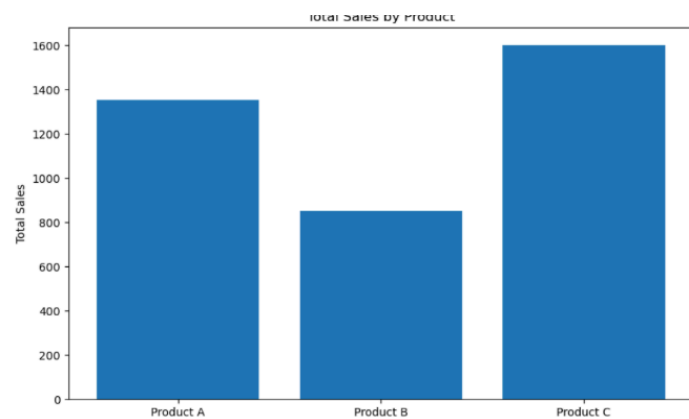
```

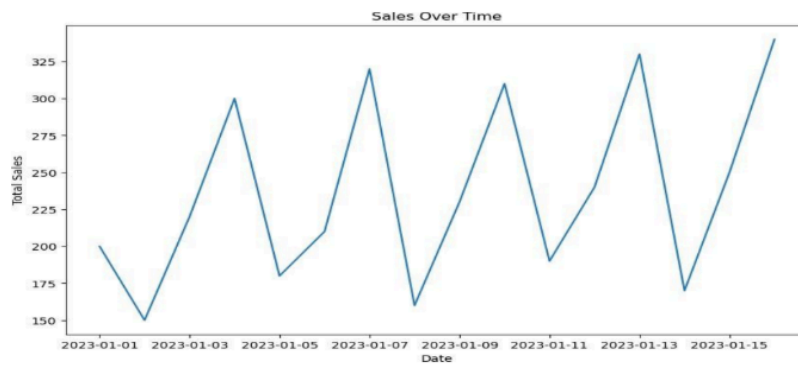
Date      0
Product   0
Sales     0
Quantity  0
Region    0
dtype: int64

```

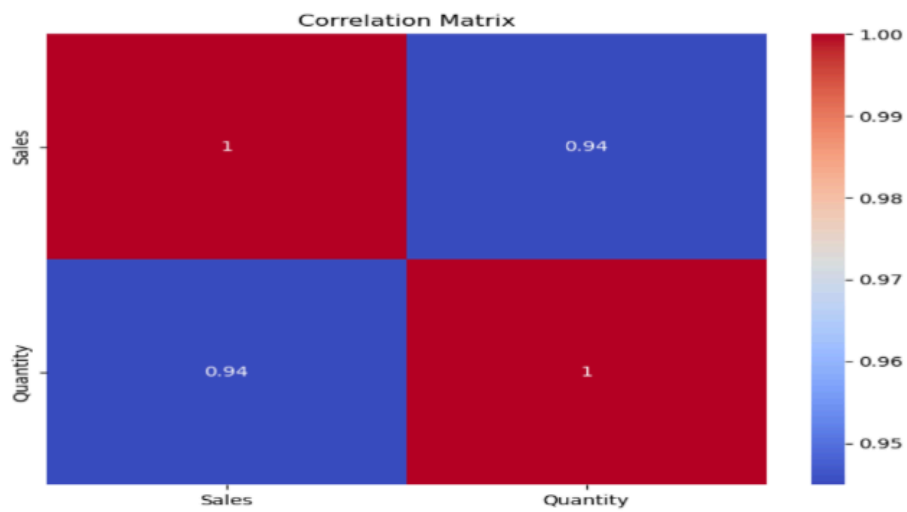
	Sales	Quantity
count	16.000000	16.000000
mean	237.500000	5.375000
std	64.031242	1.746425
min	150.000000	3.000000
25%	187.500000	4.000000
50%	225.000000	5.500000
75%	302.500000	7.000000
max	340.000000	8.000000

	Product	Sales	Quantity
0	Product A	1350	33
1	Product B	850	17
2	Product C	1600	36





	Product A	Product B	Product C
Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0
	Sales	Quantity	
Sales	1.000000	0.944922	
Quantity	0.944922	1.000000	



Exp03

fds.py - C:/Users/DEEPTHI/OneDrive/Desktop/New folder/fds.py (3.14.0)

File Edit Format Run Options Window Help

```
import numpy as np
import pandas as pd

# Load dataset
df = pd.read_csv(r"C:\Users\DEEPTHI\OneDrive\Desktop\fds obs\Hotel_Dataset.csv")

# Remove duplicates
df.drop_duplicates(inplace=True)

# Reset index properly
df.reset_index(drop=True, inplace=True)

# Drop unnecessary column
df.drop(['Age_Group.1'], axis=1, inplace=True)

# ----- FIXED LINES (NO WARNINGS) ----- #
df.loc[df['CustomerID'] < 0, 'CustomerID'] = np.nan
df.loc[df['Bill'] < 0, 'Bill'] = np.nan
df.loc[df['EstimatedSalary'] < 0, 'EstimatedSalary'] = np.nan
df.loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20), 'NoOfPax'] = np.nan
# ----- #
# Clean Hotel column
df['Hotel'] = df['Hotel'].replace(['Ibys'], 'Ibis')

# Clean FoodPreference column
df['FoodPreference'] = df['FoodPreference'].replace(['Vegetarian', 'veg'], 'Veg')
df['FoodPreference'] = df['FoodPreference'].replace(['non-Veg'], 'Non-Veg')

# Fill missing values
df['EstimatedSalary'].fillna(round(df['EstimatedSalary'].mean()), inplace=True)
df['NoOfPax'].fillna(round(df['NoOfPax'].median()), inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()), inplace=True)
df['Bill'].fillna(round(df['Bill'].mean()), inplace=True)

print(df.head())
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

```

===== FIRST 5 ROWS =====
   CustomerID Age_Group Rating(1-5) ... Bill NoOfPax EstimatedSalary
0          1.0    20-25          4 ... 1300.0      2.0      40000.0
1          2.0    30-35          5 ... 2000.0      3.0      59000.0
2          3.0    25-30          6 ... 1322.0      2.0      30000.0
3          4.0    20-25         -1 ... 1234.0      2.0     120000.0
4          5.0     35+          3 ...  989.0      2.0      45000.0

[5 rows x 8 columns]

===== DATAFRAME INFO =====
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            10 non-null    float64
1   Age_Group              10 non-null    object
2   Rating(1-5)           10 non-null    int64
3   Hotel                  10 non-null    object
4   FoodPreference         10 non-null    object
5   Bill                   10 non-null    float64
6   NoOfPax                10 non-null    float64
7   EstimatedSalary        10 non-null    float64
dtypes: float64(4), int64(1), object(3)
memory usage: 772.0+ bytes
None

```

```

===== SUMMARY STATS =====
   CustomerID Rating(1-5) Bill NoOfPax EstimatedSalary
count    10.00000    10.000000  10.000000  10.000000      10.000000
mean      5.50000     3.800000 1801.000000   2.400000     96754.700000
std       3.02765     2.250926  838.541724   0.699206     94710.847816
min       1.00000    -1.000000  989.000000   2.000000     21122.000000
25%       3.25000     3.000000 1250.500000   2.000000     41250.000000
50%       5.50000     4.000000 1561.500000   2.000000     73388.500000
75%       7.75000     5.000000 1977.250000   2.750000    114188.750000
max      10.00000     7.000000 3456.000000   4.000000    345673.000000

===== NULL VALUES =====
CustomerID      0
Age_Group       0
Rating(1-5)     0
Hotel           0
FoodPreference  0
Bill            0
NoOfPax         0
EstimatedSalary 0
dtype: int64

```

Exp 4

```
import numpy as np
import pandas as pd
df=pd.read_csv("/pre_process_datasample - pre_process_datasample.csv")
df.info()
df.Country.mode()
df.Country.mode()[0]
type(df.Country.mode())
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
pd.get_dummies(df.Country)
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,[1,2,3]]],axis=1)
df.info()
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Country     10 non-null     object
1   Age         9 non-null      float64
2   Salary      9 non-null      float64
3   Purchased   10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Country     10 non-null     object
1   Age         10 non-null     float64
2   Salary      10 non-null     float64
3   Purchased   10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
/tmp/ipython-input-1095748479.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series t
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which
```



```
df.Country.fillna(df.Country.mode()[0],inplace=True)
tmp/ipython-input-1095748479.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series th
he behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which

or example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or

df.Age.fillna(df.Age.median(),inplace=True)
tmp/ipython-input-1095748479.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series t
he behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which

or example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or

df.Salary.fillna(round(df.Salary.mean()),inplace=True)
tmp/ipython-input-1095748479.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series t
he behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which

or example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or

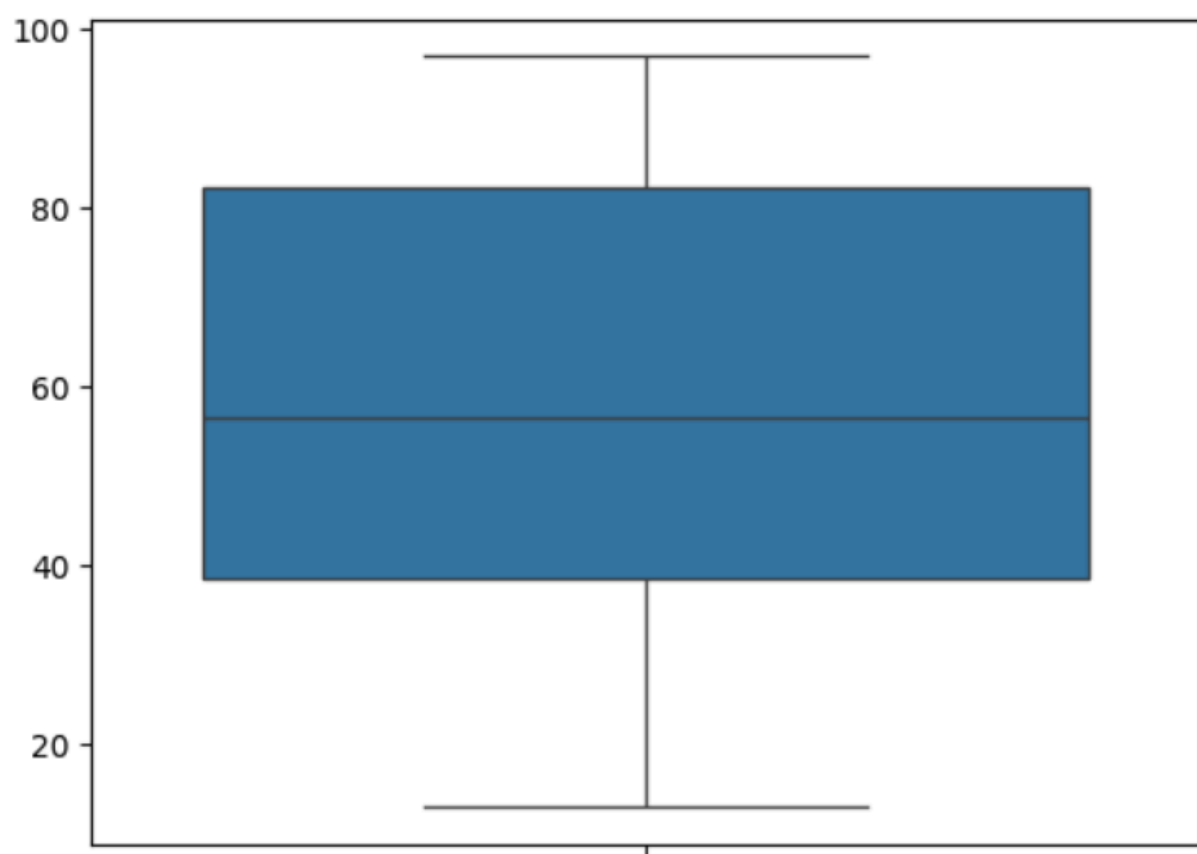
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
tmp/ipython-input-1095748479.py:14: FutureWarning: Downcasting behavior in `replace` is deprecated and will be rem
updated_dataset.Purchased.replace(['No','Yes'],[0,1],inplace=True)
```

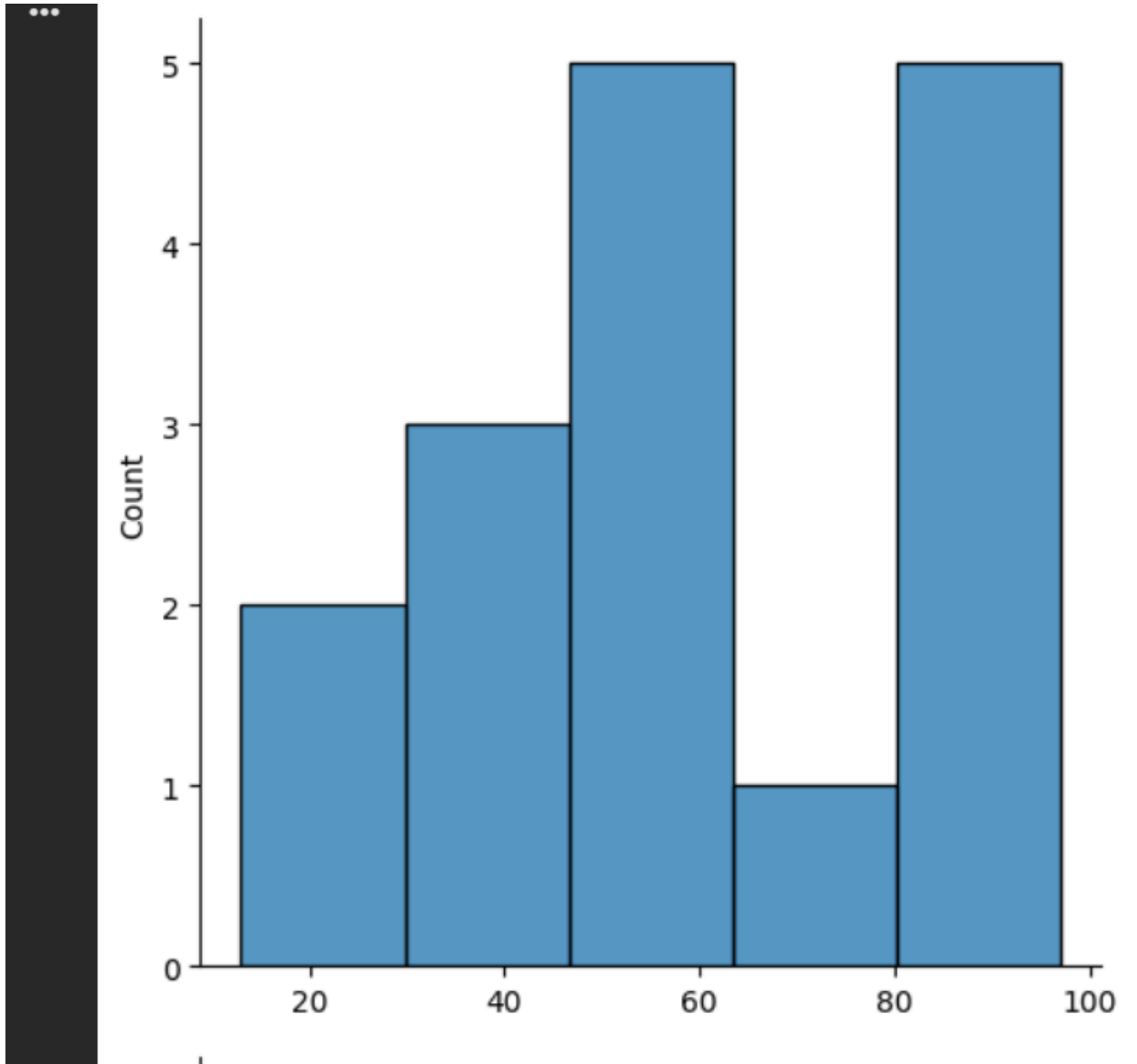
```
import numpy as np
array = np.random.randint(1,100,16)
array.mean()

np.percentile(array,25)
np.percentile(array,50)
np.percentile(array,75)
np.percentile(array,100)
def outDetection(array):
    sorted(array)
    q1,q3 = np.percentile(array,[25,75])
    iqr = q3-q1
    lr = q1 - (1.5*iqr)
    ur = q3 + (1.5*iqr)
    return lr,ur
lr,ur = outDetection(array)

import seaborn as sns
sns.boxplot(array)
%matplotlib inline
sns.displot(array)
new_array = array[(array>lr) & (array<ur)]
sns.displot(new_array)
lr1,ur1 = outDetection(new_array)
final_array = new_array[(new_array>lr1) & (new_array<ur1)]
sns.displot(final_array)
sns.displot(final_array)
```

<seaborn.axisgrid.FacetGrid at 0x794aeb7054c0>





Exp08

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
df.info()
df.dropna(inplace=True)
df.info()
df.describe()
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(features, label, test_size=0.2, random_state=42)
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
model.score(x_train,y_train)
model.score(x_test,y_test)
model.coef_
model.intercept_
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
model=pickle.load(open('SalaryPred.model','rb'))
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}: ".format(yr_of_exp,Salary))
```

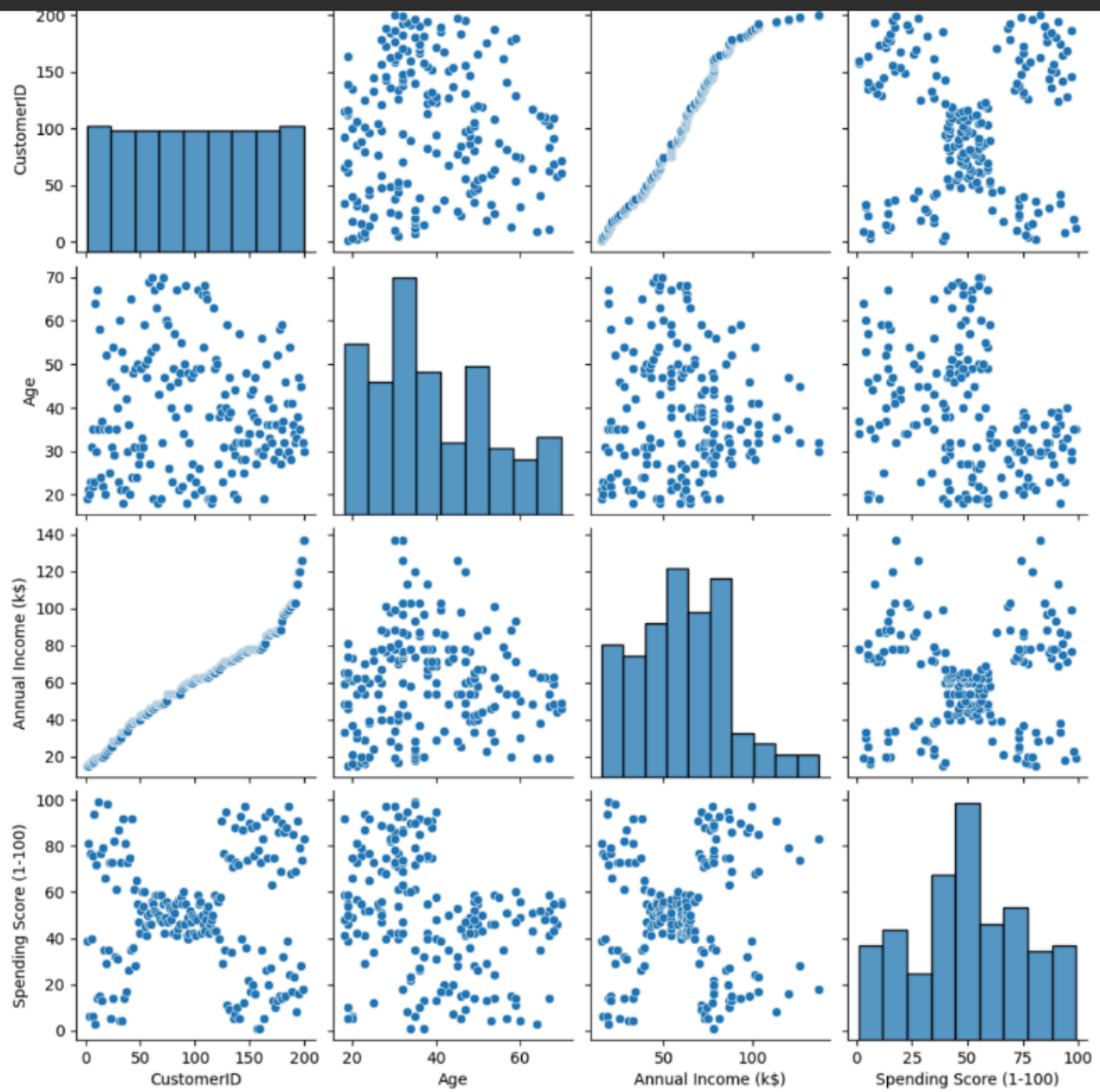
Exp09

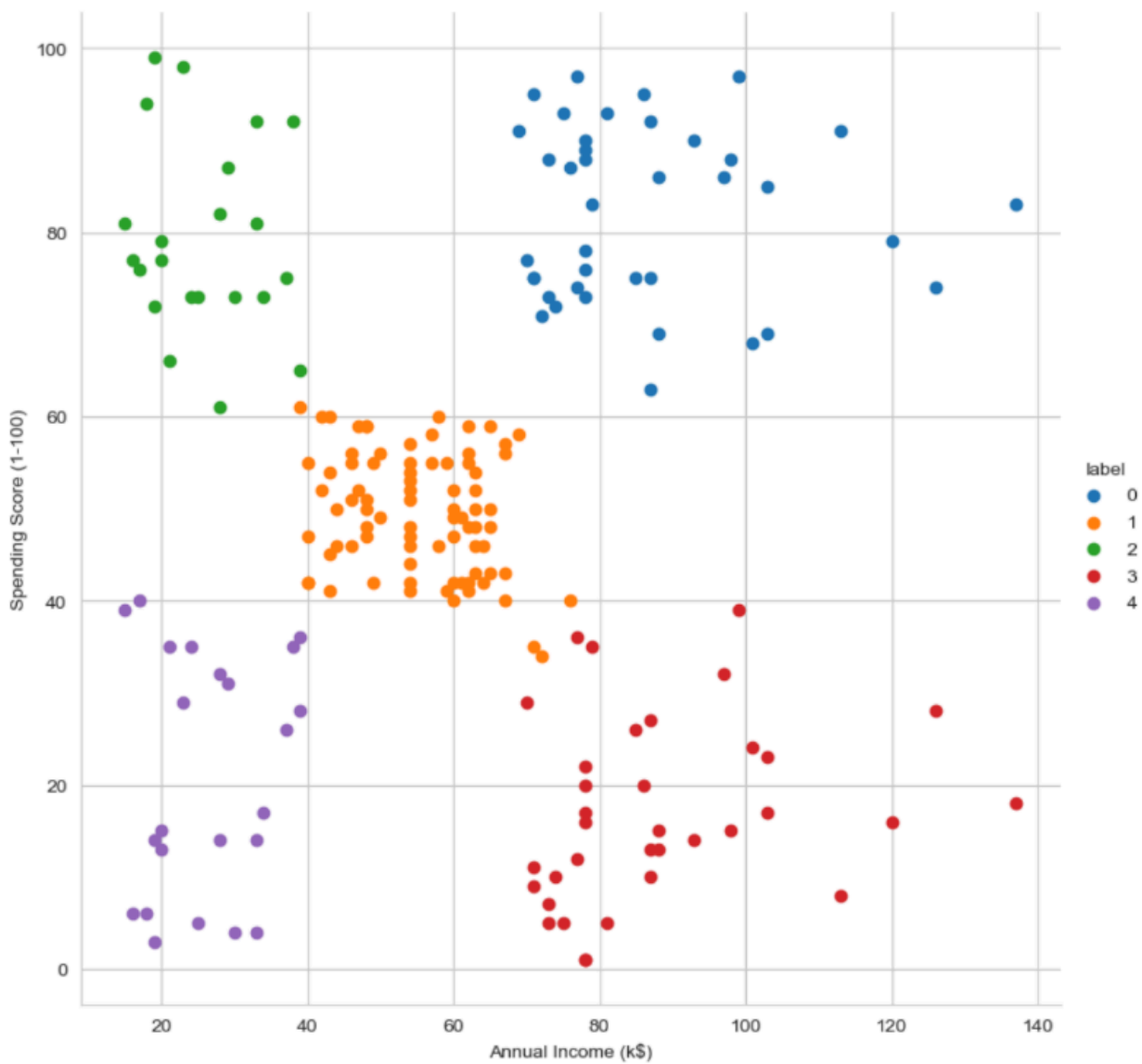
```
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df.head()
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
for i in range(1,401):
    x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2, random_state=i)
    model=LogisticRegression()
    model.fit(x_train,y_train)
    train_score=model.score(x_train,y_train)
    test_score=model.score(x_test,y_test)
    if test_score>train_score:
        print("Test {} Train{} Random State {}".format(test_score,train_score,i))
```

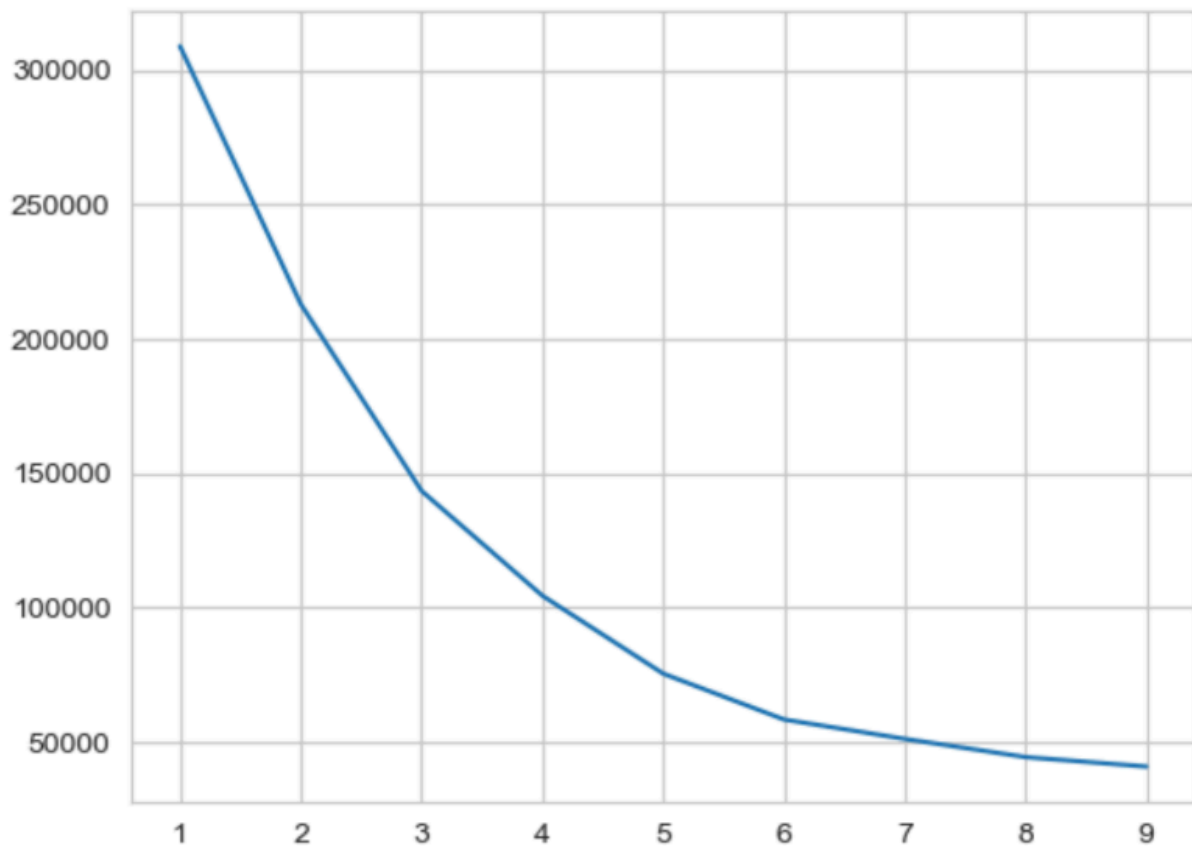
Exp10

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
df=pd.read_csv('Mall_Customers.csv')
df.info()
df.head()
sns.pairplot(df)
features=df.iloc[:,[3,4]].values
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
model=KMeans(n_clusters=i)
model.fit(features_el)
wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```









Exp 11

```
import numpy as np
import pandas as pd
df=pd.read_csv('Iris.csv')
df.info()
df.variety.value_counts()
df.head()
features=df.iloc[:, :-1].values
label=df.iloc[:, 4].values
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state = )
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

	precision	recall	f1-score	support
Setosa	1.00	1.00	1.00	50
Versicolor	0.96	0.94	0.95	50
Virginica	0.94	0.96	0.95	50
accuracy			0.97	150
macro avg	0.97	0.97	0.97	150
weighted avg	0.97	0.97	0.97	150

Exp 12

fds.py - C:/Users/DEEPTHI/OneDrive/Desktop/New folder/fds.py (3.14.0)

File Edit Format Run Options Window Help

```
import numpy as np
from scipy import stats
# Sample data
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
# Hypothesized mean
mu_0 = 70
# One-sample t-test
t_stat, p_value = stats.ttest_1samp(marks, mu_0)
print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value > alpha:
    print("Reject Null Hypothesis -> Mean is significantly different from 70.")
else:
    print("Fail to Reject Null Hypothesis -> No significant difference.")
```

```
===== RESTART: C:/Users/DEEPTHI/OneDrive/Desktop/New folder/fds.py
T-statistic: 1.993
P-value: 0.0774
Reject Null Hypothesis -> Mean is significantly different from 70.
```

Exp 13

```
fds.py - C:/Users/DEEPTHI/OneDrive/Desktop/New folder/fds.py (3.14.0)
File Edit Format Run Options Window Help

import numpy as np
from math import sqrt
from scipy.stats import norm
#given data
x_bar = 51.2 #avgwt
mu_0 = 50
sigma = 3
n = 36
#calc z-statistic
z_stat = (x_bar - mu_0) / (sigma/sqrt(n))

p_value = 2*(1-norm.cdf(abs(z_stat)))
print(f"Z-statistic:{z_stat:.3f}")
print(f"P-value:{p_value:.4f}")
alpha = 0.05
if p_value < alpha:
    print("Reject null hypothesis-> Mean is significantly different from 50g.")
else:
    print("Fail to reject Null hypothesis -> no significant difference.")
```

```
===== RESTART: C:/Users/DEEPIHI/OneDrive/Desktop/New folder/tds.py (3.14.0) =====  
Z-statistic:2.400  
P-value:0.0164  
Reject null hypothesis-> Mean is significantly different from 50g.
```

Exp 14

tds.py - C:/Users/DEEPIHI/OneDrive/Desktop/New folder/tds.py (3.14.0)

File Edit Format Run Options Window Help

```
#anova test  
import numpy as np  
from scipy import stats  
a = [20,22,23]  
b = [19,20,18]  
c = [25,27,29]  
#one way anova  
f_stat,p_value = stats.f_oneway(a,b,c)  
print(f"F-Statistic:{f_stat:.3f}")  
print(f"P-value:{p_value:.4f}")  
alpha = 0.05  
if p_value < alpha:  
    print("Reject null hypothesis -> means are significantly different.")  
else:  
    print("Fail to reject null hypothesis -> no significant difference.")
```

```
===== RESTART: C:/Users/DEEPIHI/OneDrive/Desktop/New folder/tds.py (3.14.0) =====  
F-Statistic:20.364  
P-value:0.0021  
Reject null hypothesis -> means are significantly different.
```